Orchestrazione SDN di reti QKD multi-dominio

L'interfaccia ETSI GS QKD 018

Diego Carro Bucci Felice Elia

Corso di Cybersecurity, Politecnico di Torino
Quantum Engineering

Novembre 2026

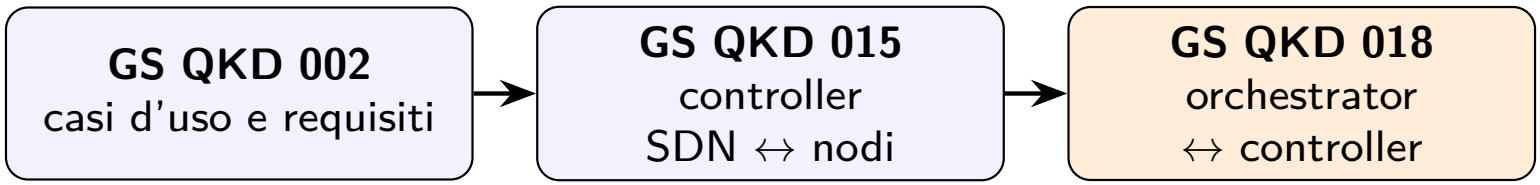
Indice

1. Introduzione: dalla QKD alla rete
2. ETSI GS QKD 015: il controller SDN
3. ETSI GS QKD 018: l'interfaccia di orchestrazione
4. Casi reali, limiti e conclusioni

1. Introduzione: dalla QKD alla rete
   - Motivazione
   - QKD e trusted node
   - Architettura e SDN
   - GS QKD 002
2. ETSI GS QKD 015: il controller SDN
3. ETSI GS QKD 018: l'interfaccia di orchestrazione
4. Casi reali, limiti e conclusioni

Il problema

- La QKD permette a due parti di concordare una chiave segreta con sicurezza fondata sulla fisica, non sulla difficoltà computazionale.
- Portarla in una rete di operatore significa gestirla come qualunque altra risorsa di rete:
  - scoprirne la topologia e monitorarne lo stato;
  - allocarne la capacità ai servizi;
  - coordinarla con la rete ottica classica.
- L'ETSI ISG QKD ha affrontato il problema per gradi:



Stato dell'arte in breve

Fondamenti e reti

- Review dei protocolli e delle prove di sicurezza [1]
- Dal link alla rete: strati, trusted relay, instradamento [2–4]

Standard

- ETSI: casi d'uso [5], API [6, 7], SDN [8], orchestrazione [9], KMS [10]
- ITU-T Y.3800: architettura, controllo, SDN [11–13]

SDN e orchestrazione

- Primi esperimenti SDN + QKD (UPM/Telefónica) [14, 15]
- KOREN multi-vendor (SK Telecom) [16]
- Prototipo gerarchico ONOS (PoliMi) [17]
- MadQCI, Madrid [18]

Contributo

Lettura unitaria della catena 002 $\rightarrow$ 015 $\rightarrow$ 018: dai requisiti ai modelli YANG all'orchestrazione multi-dominio.

La distribuzione quantistica di chiavi

BB84 in quattro passi

1. Alice codifica bit casuali su singoli fotoni, in una di due basi scelte a caso.
2. Bob misura in una base casuale.
3. Su un canale classico autenticato confrontano le basi (*sifting*).
4. Correzione d'errore e *privacy amplification* distillano la chiave.

Un intercettatore introduce errori misurabili:

$$\mathrm{QBER} = \frac{N_{\text{errori}}}{N_{\text{sifted}}} \lesssim 11\% \text{ (BB84)}$$

Ciò che interessa alla rete

Il *secret key rate* (SKR), in bit/s, cala con la distanza:

$$\eta = 10^{-\alpha L/10}$$

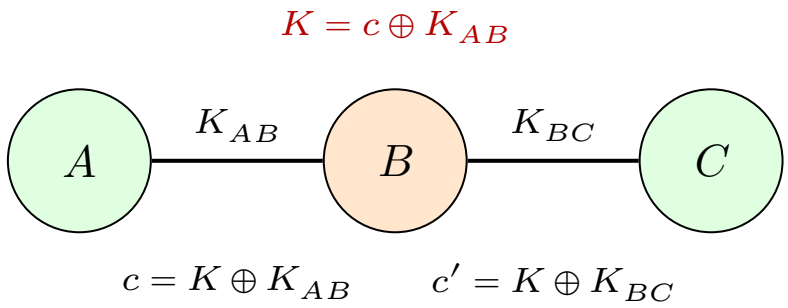
con $\alpha \approx 0{,}2$ dB/km.

Due osservazioni

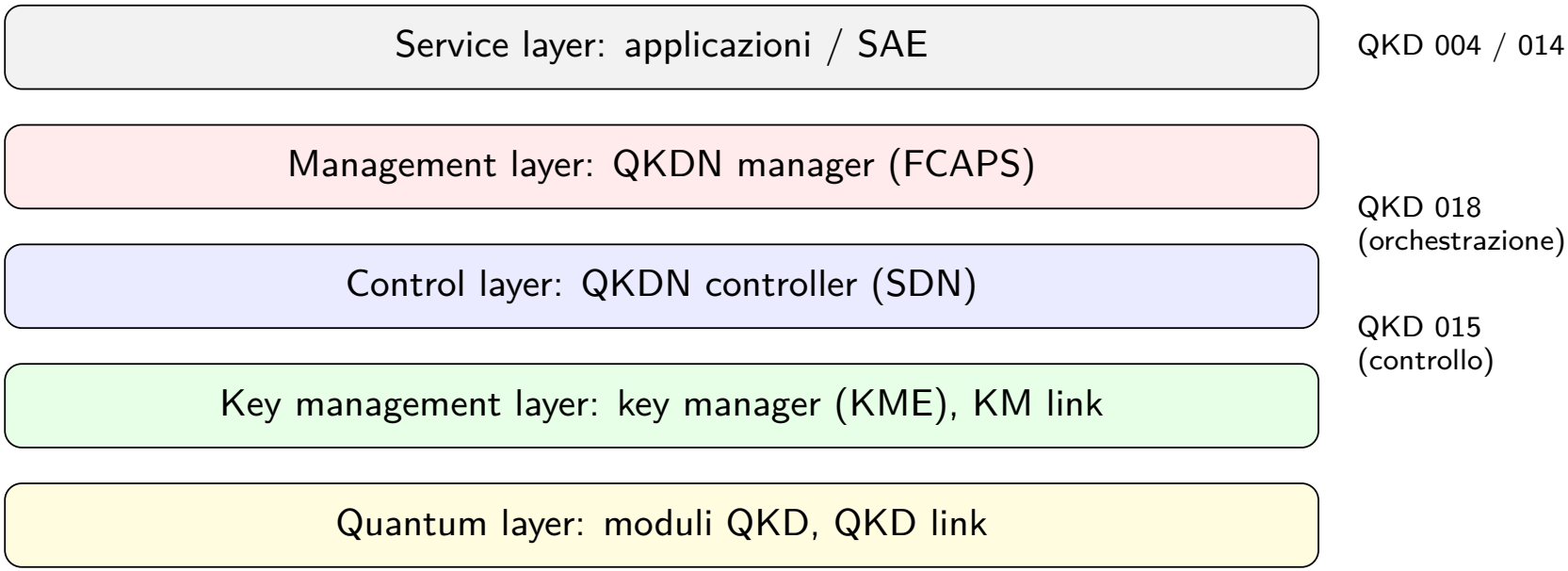
- La QKD *distribuisce* chiavi, non cifra né autentica.
- Serve un segreto iniziale: è *key growing*.

Dal link alla rete: il trusted node

- Un link copre al più qualche centinaio di km; non esistono ancora ripetitori quantistici pratici.
- Le reti reali collegano i link in serie tramite nodi fidati.
- Ogni hop è un one-time pad, ma K è in chiaro dentro ogni nodo intermedio.
- Il relay consuma chiave: ogni link $\ell \in p$ spende $|K|$ bit. La chiave diventa una risorsa da pianificare.



L'architettura a strati ITU-T Y.3800



QKD link: due moduli QKD, dove la chiave è *generata*.
KM link: due key manager, dove la chiave è *inoltrata*.

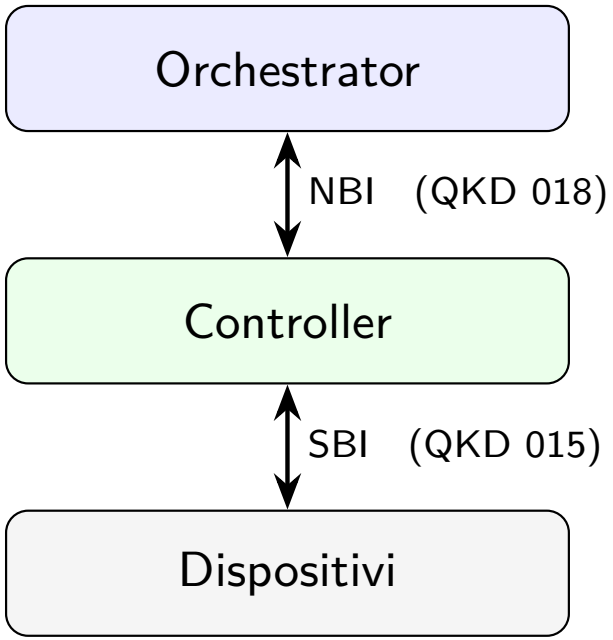
QKDN controller: decide in tempo reale (cammini, link).
QKDN manager: supervisiona (FCAPS, politiche).

Adattata da ITU-T Y.3800 [11]

Software-Defined Networking

- SDN separa *piano dati*, *piano di controllo* (controller con vista globale) e *piano applicativo* [19].
- Il controller parla ai dispositivi con la SBI e offre servizi verso l'alto con la NBI.
- Con più domini si aggiunge un orchestrator.

Una rete QKD ha tutto ciò che rende utile SDN: risorse scarse e variabili, vincoli fisici, apparati multi-vendor, domanda che cambia.



GS QKD 002 (2010): dove serve la QKD

Una primitiva tra tre

Un sistema sicuro richiede *cifratura*, *distribuzione delle chiavi*, *autenticazione*. La QKD copre solo la seconda: è un complemento, non un sostituto, della crittografia classica (OTP/AES, MAC Wegman–Carter).

Dove si consuma la chiave

- L2 PPP, MACsec, QKD link encryptor
- L3 IKE modificato per IPsec
- L4 chiavi di sessione TLS, HMAC
- L7 autenticazione e cifratura applicativa

In ogni caso c'è chi *consuma* e chi *fornisce* chiavi: la futura separazione SAE / KME.

[5]

I sei casi d'uso

Caso	Configurazione	Esigenza che anticipa
UC1 Backup	link punto-punto + cifratore di linea	interfaccia chiave–cifratore
UC2 Metro aziendale	più link, server di chiavi	QoS per applicazione
UC3 Infrastrutture critiche	rete con key relay e gestione integrata	KMS, ridondanza, controllo centralizzato
UC4 Backbone	canale quantistico in WDM	coesistenza con la rete ottica, gestione integrata
UC5 Accesso PON	punto-multipunto	condivisione della capacità di chiave
UC6 Long-haul	relay tramite nodo mobile (satellite)	trusted node, ridondanza

Che cosa lascia in eredità

La QKD utile è una *rete*: gestione delle chiavi (UC3, UC6) e QoS differenziata (UC2) $\Rightarrow$ **GS QKD 015**; integrazione con la rete ottica dell'operatore (UC4) $\Rightarrow$ **GS QKD 018**.

1 Introduzione: dalla QKD alla rete

2 ETSI GS QKD 015: il controller SDN
- Componenti
- Modello YANG
- Workflow e sicurezza

3 ETSI GS QKD 018: l'interfaccia di orchestrazione

4 Casi reali, limiti e conclusioni

Perché un controllo software-defined

Oggi: integrazione manuale

- ogni sistema installato a mano su fibra scura dedicata;
- non scala;
- la capacità di chiave non si sposta dove serve;
- non dialoga con la rete ottica.

Obiettivi dell'astrazione

- gestione centrale delle risorse QKD;
- un'unica gestione delle chiavi per perimetro;
- parametri di basso livello delegati al nodo;
- link distribuiti secondo la QoS;
- coordinamento di canali quantistici e classici.

GS QKD 015 V2.1.1 (2022) [8]: la southbound interface del piano di controllo QKD, allineata alla 018.

Le componenti di una rete SD-QKD

Modulo QKD hardware + software, specifico del vendor

Interfaccia QKD `qkdi`: vista astratta di un modulo (tecnologia, modello, λ, attacco ottico)

Nodo SD-QKD moduli nello stesso luogo sicuro, un solo *SDN agent*, un key manager, un punto di accesso alle chiavi

Link `qkd_link`: `PHYS` (canale quantistico diretto) o `VIRT` (key relay)

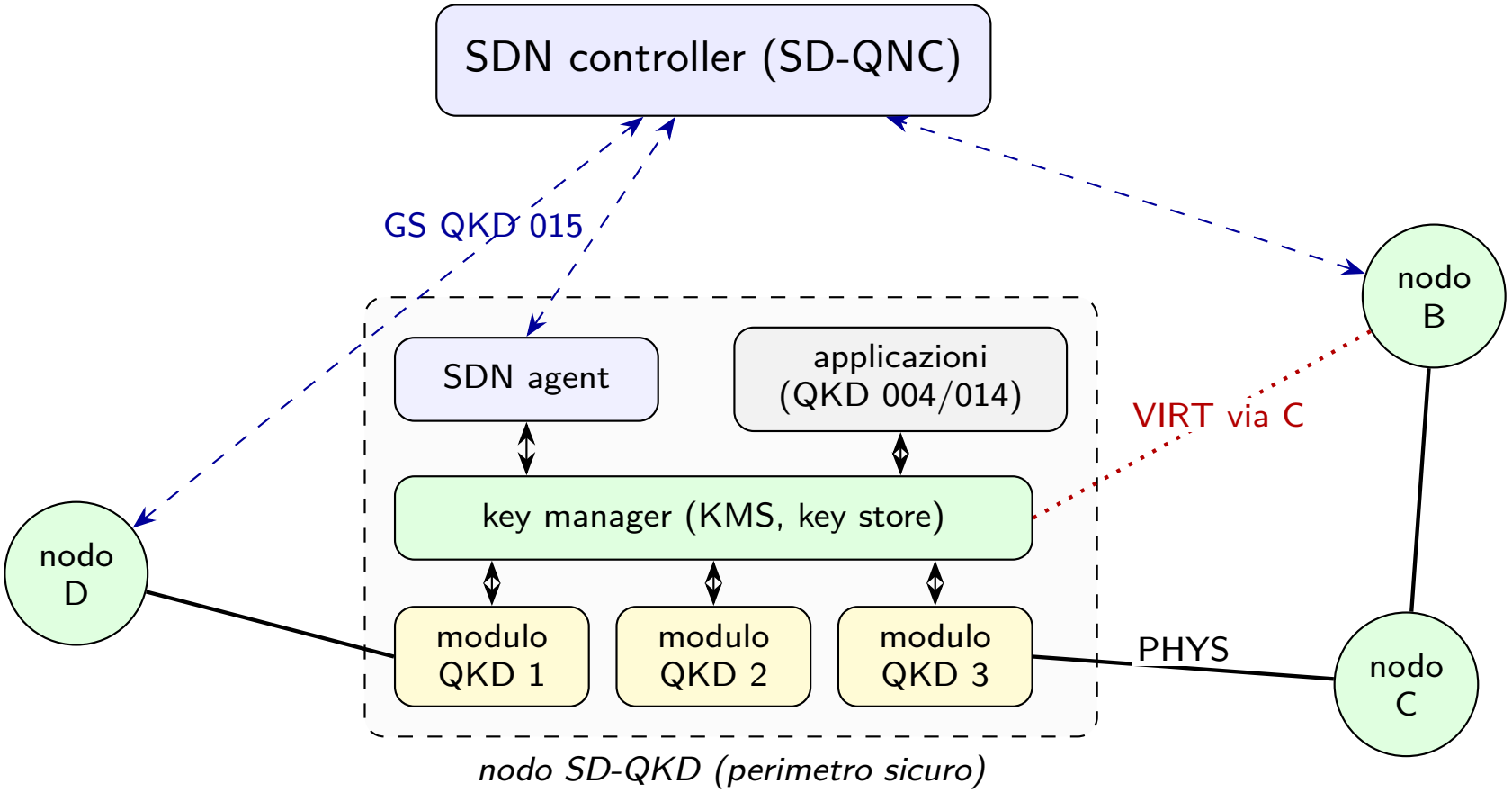
Applicazione `qkd_app`: `CLIENT` o `INTERNAL`, con QoS

Controller SD-QNC: registro delle app, pianifica i link, dialoga con la rete ottica (SD-ONC)

Principio di sicurezza

Il materiale di chiave *non lascia mai il perimetro del nodo*: al controller salgono solo identificativi, stati, tassi e parametri fisici.

Il nodo SD-QKD



Adattata da GS QKD 015, Fig. 1 [8]

Link virtuali e applicazioni

Key association link `VIRT`

- consuma chiave dai link fisici sottostanti, quindi è un'*applicazione interna* con `virt_bandwidth`;
- ogni nodo intermedio conosce `virt_prev_hop` e `virt_next_hop`;
- richiede `key_relay_mode_enable`.

QoS di un'applicazione

- `max_bandwidth`, `min_bandwidth` (bit/s)
- `jitter` massimo (ms)
- `ttl` della chiave nel key store
- condivisione di cammini o chiavi

È la risposta all'esigenza di QoS differenziata di UC2.

Anatomia di un modello YANG

YANG [20] è uno schema (come XSD o JSON Schema) dei dati di configurazione e stato e delle operazioni.

- `leaf`, `leaf-list`, `container`, `list` con `key`
- `config true` (decide il controller) / `config false` (misura il nodo)
- `grouping`/`uses` per riusare, `augment` per estendere un altro modulo
- `leafref`, `when`, `must` per vincoli
- `rpc` e `notification`

YANG: intestazione del modulo e identity

```
module etsi-qkd-sdn-node {
  namespace "urn:etsi:qkd:yang:etsi-qkd-node";
  prefix "etsi-qkdn";
  import etsi-qkd-node-types { prefix "etsi-qkdn-types"; }
  revision "2022-01-30" { reference "ETSI␣GS␣QKD␣015␣V2.1.1"; }
  ...
}
identity QKD-LINK-TYPES { description "QKD␣key␣association␣link␣types."; }
identity VIRT { base QKD-LINK-TYPES; }
identity PHYS { base QKD-LINK-TYPES; }
typedef qkd-link-types {
  type identityref { base QKD-LINK-TYPES; }
}
```

Le `identity` sono estendibili da altri moduli, a differenza di un'`enumeration`.

Il modello del nodo: etsi-qkd-sdn-node

```
module: etsi-qkd-sdn-node
  +--rw qkd_node
     +--rw qkdn_id                yang:uuid
     +--ro qkdn_status?           etsi-qkdn-types:qkdn-status-types
     +--rw qkdn_capabilities
     |  +--rw link_stats_support?  application_stats_support?
     |  +--rw key_relay_mode_enable?
     +--rw qkd_applications
     |  +--rw qkd_app* [app_id]
     |     +--rw app_id, app_type, server_app_id, client_app_id*
     |     +--rw app_qos             (max/min_bandwidth, jitter, ttl)
     |     +--rw backing_qkdl_id*, local_qkdn_id, remote_qkdn_id
     +--rw qkd_interfaces
     |  +--rw qkd_interface* [qkdi_id]
     |     +--rw qkdi_model, qkdi_type, qkdi_att_point
     |     +--rw qkdi_capabilities  (role, wavelength_range, ...)
     +--rw qkd_links
        +--rw qkd_link* [qkdl_id]
           +--rw qkdl_type, qkdl_enable, qkdl_local / qkdl_remote
           +--rw virt_prev_hop, virt_next_hop*, virt_bandwidth (VIRT)
           +--rw phys_channel_att, phys_wavelength          (PHYS)
           +--rw qkdl_performance (skr, eskr, expected_consumption)
```

Il modello del nodo: sezioni e prestazioni

Un albero per nodo, quattro sezioni:

- identità e **capability**;
- **applicazioni** con QoS;
- **interfacce** (moduli);
- **link** con prestazioni.

Prestazioni del link ℓ

$$\mathrm{EC}_\ell = \sum_{a \in \mathcal{A}_\ell} b_a$$

$$\mathrm{ESKR}_\ell \approx \mathrm{SKR}_\ell - \mathrm{EC}_\ell$$

EC = `expected_consumption`; lettura intuitiva, non formula dello standard.

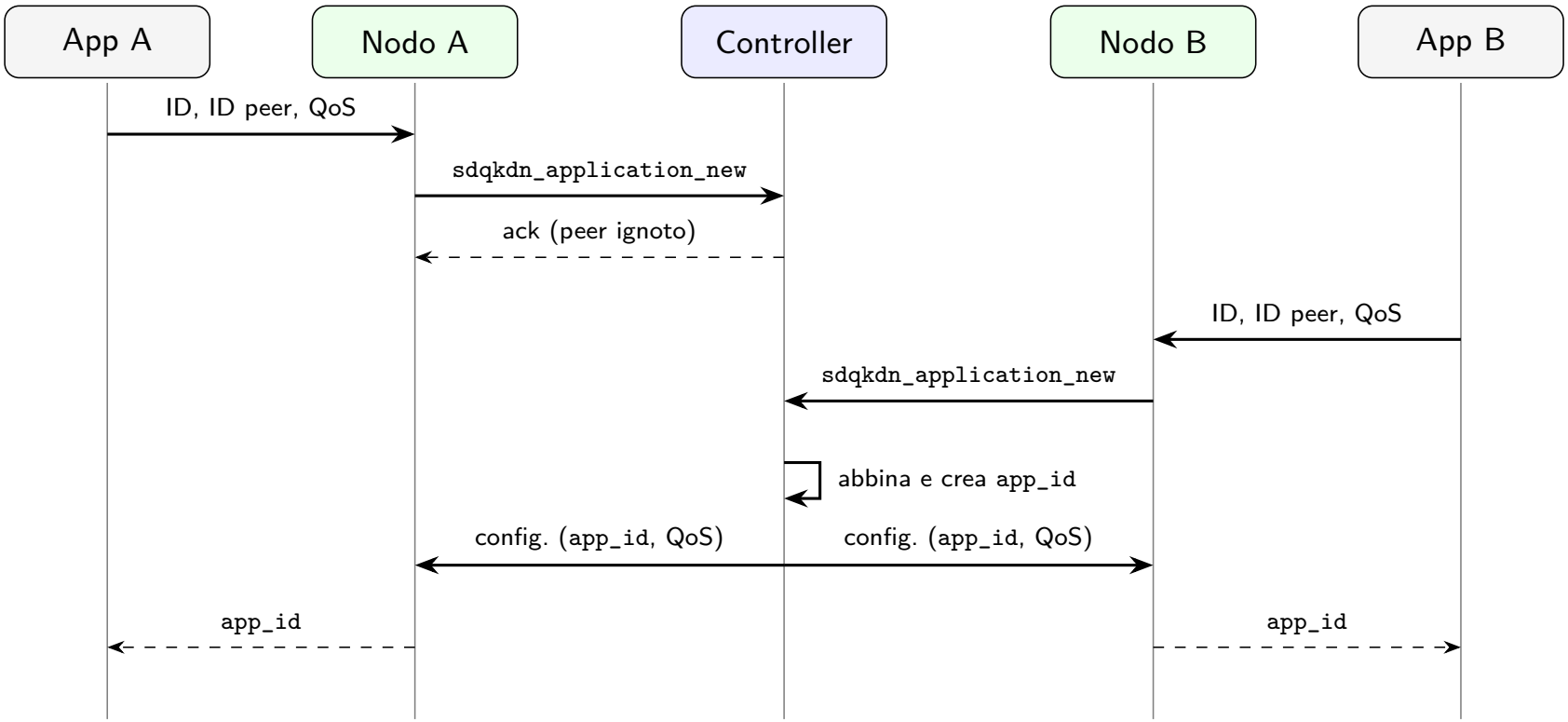
Notifiche dal nodo al controller

Interrogare continuamente i nodi è inefficiente: la 015 definisce notifiche YANG per oggetto.

Oggetto	**Notifiche**
Applicazioni	`sdqkdn_application_new`, `_qos_update`, `_disconnected`
Interfacce	`sdqkdn_interface_new`, `_down` (con *reason*), `_out`
Link	`sdqkdn_link_down`, `_perf_update`, `_overloaded`
Generiche	eventi e allarmi estendibili

Inoltre NETCONF/RESTCONF permettono di sottoscrivere qualunque dato del datastore e ricevere ogni variazione in tempo reale.

Workflow: registrazione di un'applicazione



Le applicazioni conoscono solo il proprio key manager: è il controller, registro centrale, ad abbinare le richieste e creare l'app_id.

Adattata da GS QKD 015, Fig. 2 [8]

Workflow: creazione di link fisici e virtuali

Link fisico

1. innesco (domanda sui nodi o NBI);
2. due interfacce libere e compatibili;
3. richiesta di un *lightpath* all'SD-ONC;
4. configurazione delle interfacce.

Cuore della coesistenza con la rete ottica (UC4).

Link virtuale

Se manca un lightpath: cammino sulla rete QKD, nodi intermedi in modalità relay, estremi con la banda richiesta. Esempio (nostro, non dello standard):

$$w_\ell = \frac{1}{\mathrm{ESKR}_\ell}$$

$$p^\star = \arg\min_{p \in \mathcal{P}_r} \sum_{\ell \in p} w_\ell$$

$$\mathcal{P}_r = \left\{ p : \min_{\ell \in p} \mathrm{ESKR}_\ell \geq r \right\}$$

Link virtuali mal gestiti congestionano i link fisici: serve la vista globale del controller.

Protocolli, sicurezza e limiti della 015

Protocolli

- NETCONF [21]: apparato–controller, transazionale, SSH/TLS, XML
- RESTCONF [22]: NBI, HTTP, CRUD, JSON/XML

Installazione sicura

- area sicura esistente: canale protetto con chiavi QKD;
- nuovo sito: segreto iniziale, poi chiavi QKD (*key growing*).

Che cosa resta fuori

La 015 controlla *un* dominio con *un* controller.

- più domini QKD, anche di vendor diversi;
- SAE nei nodi di un'altra rete (OTN);
- gestione con gli stessi strumenti di orchestrazione del resto della rete.

$\Rightarrow$ **GS QKD 018**

1 Introduzione: dalla QKD alla rete

2 ETSI GS QKD 015: il controller SDN

3 ETSI GS QKD 018: l'interfaccia di orchestrazione
- Scenario e modello
- Moduli YANG
- Workflow e sicurezza
- Analogia ACTN

4 Casi reali, limiti e conclusioni

GS QKD 018 V1.1.1 (2022)

- Interfaccia tra SDN orchestrator (client) e SDN controller del dominio QKD (server) [9].
- È la *northbound interface* del controller: il complemento della 015.

$$\text{orchestrator} \xrightarrow{\text{QKD 018}} \text{controller} \xrightarrow{\text{QKD 015}} \text{nodo SD-QKD} \xrightarrow{\text{QKD 004/014}} \text{SAE}$$

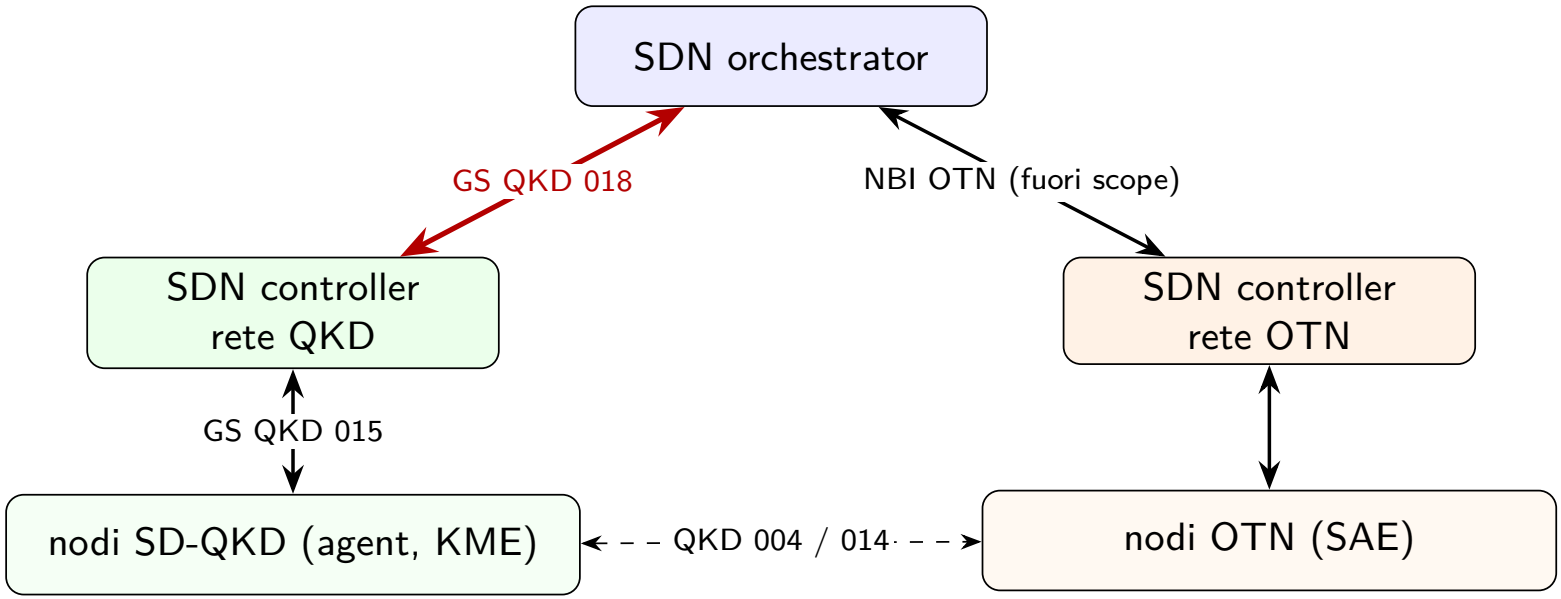
Orchestrazione SDN

Coordinamento automatico e continuo delle risorse per stabilire e rilasciare servizi end-to-end che attraversano più domini, ciascuno con il proprio controller.

Service link

Associazione logica di chiave tra due nodi QKD, single- o multi-hop: a livello di orchestrazione è il link virtuale della 015.

Lo scenario di riferimento



- Rete QKD separata dalla rete di trasporto ottico (OTN): prestazioni, isolamento, responsabilità distinte.
- I SAE stanno nei nodi OTN, le chiavi nei nodi QKD; ogni controller vede solo la propria rete.

L'orchestrator risolve:

1. *address matching* nodo OTN $\leftrightarrow$ nodo QKD;
2. provisioning end-to-end;
3. gestione: topologia, inventario, FCAPS.

Adattata da GS QKD 018, Fig. 1 [9]

Funzioni dell'orchestrator e modello informativo

Funzioni richieste (§4.2)

- tradurre le richieste in configurazione dei domini;
- attivare e rilasciare servizi inter-dominio;
- calcolare e modificare cammini multi-dominio;
- scoprire nodi e link, vista astratta;
- monitorare le risorse di chiave e FCAPS.

Cinque blocchi informativi (§5)

§5.1 discovery della topologia, proattiva o reattiva

§5.2 stato e inventario: SKR, ESKR, ripristino

§5.3 stato dei servizi end-to-end

§5.4 provisioning con calcolo dei cammini candidati

§5.5 notifiche e soglie configurabili (QBER, SKR, ESKR)

Ammissibilità di un cammino (formalizzazione nostra)

Ogni link di un cammino multi-hop inoltra la stessa chiave, quindi conta il collo di bottiglia:

$$p \text{ ammissibile per la richiesta } r \iff \min_{\ell \in p} \mathrm{ESKR}_\ell \geq r$$

Tre moduli YANG: topology

```
qkd_network
 +-- sdqkd_nodes
 |    +-- qkdn* [qkdn_id]                (nodi SD-QKD)
 |         +-- qkd_interfaces
 |              +-- qkdi* [qkdi_id]      (moduli QKD)
 +-- qkd_links
      +-- qkd_phys_links
      |    +-- phys_link* [phys_link_id]
      |         +-- link_type
      |         +-- phys_link_local_qkdn
      |         |    (qkdn_id_ref, qkdi_id_ref)
      |         +-- phys_link_remote_qkdn
      +-- qkd_svc_links
           +-- svc_link* [svc_link_id]
                +-- ... estremi locale/remoto
```

Catena: *topology* → *inventory* → *connectivity*. I moduli importano i tipi della 015.

`etsi-qkd-sdn-topology`: un solo albero per l'intero dominio, cioè un grafo

$$G_i = (V_i, E_i), \quad E_i = E_{\text{phys}} \cup E_{\text{svc}}$$

con estremi `leafref` validati. L'orchestrator compone la vista multi-dominio:

$$G = \left(\bigcup_i V_i,\ \bigcup_i E_i \cup E_{\text{inter}} \right)$$

E_{inter}: adiacenze tra domini, tipicamente nodi di bordo.

[23]

Tre moduli YANG: inventory

`etsi-qkd-sdn-inventory` non crea un nuovo albero: fa `augment` della topologia.

Elemento	**Foglie aggiunte**
`qkdn`	versione, capability, location, `KME_ip_port` (dove i SAE prelevano le chiavi via QKD 014), stato
`qkdi`	`role_support`, modello, tipo, punto di attacco, stato
`phys_link`	`enable`, `wavelength`, `performance` (`skr`, `eskr`, `expected_consumption`, `phys_link_perf`: QBER, SNR), stato
`svc_link`	`bandwidth`, `performance`, `path_restoration` (`AUTOMATIC`/`MANUAL`, `hold_off_time`), stato

Notifiche: `sdqkdn_new`, `_interface_new`, `_phys_link_new`, `_svc_link_new`, varianti `_perf_update` e `_overloaded` per link fisici e service link, `alarm` ed `event` (lo stato del nodo viaggia qui).

Tre moduli YANG: connectivity

```
connectivity_service/qkd_service
 +-- qkd_app* [app_id]
      +-- (dati app da QKD 015: stato, QoS)
      +-- app_mapped_svc_link
           +-- svc_link_and_qkdn_ids
           +-- path_constraints
           |    (include/exclude_nodes:
           |     coppie nodo, interfaccia)
           +-- candidate_paths
           |    +-- candidate_path* [path_id]
           +-- deploy_path (path_id, phys_links)
```

Divisione dei ruoli

Il controller conosce il dominio e propone i cammini; l'orchestrator ha la vista d'insieme e decide.

RPC: il workflow di provisioning passo per passo

1 app-registration
2 create-service-link-id-for-app
3 create-candidate-paths-per-service-link-for-app
3' update-candidate-paths-per-service-link-for-app
4 deploy-a-candidate-path-per-service-link-for-app
5 delete-deployed-path-per-service-link-for-app

Notifiche

sdqkdn_application_new, _deployed, _qos_update, _disconnected

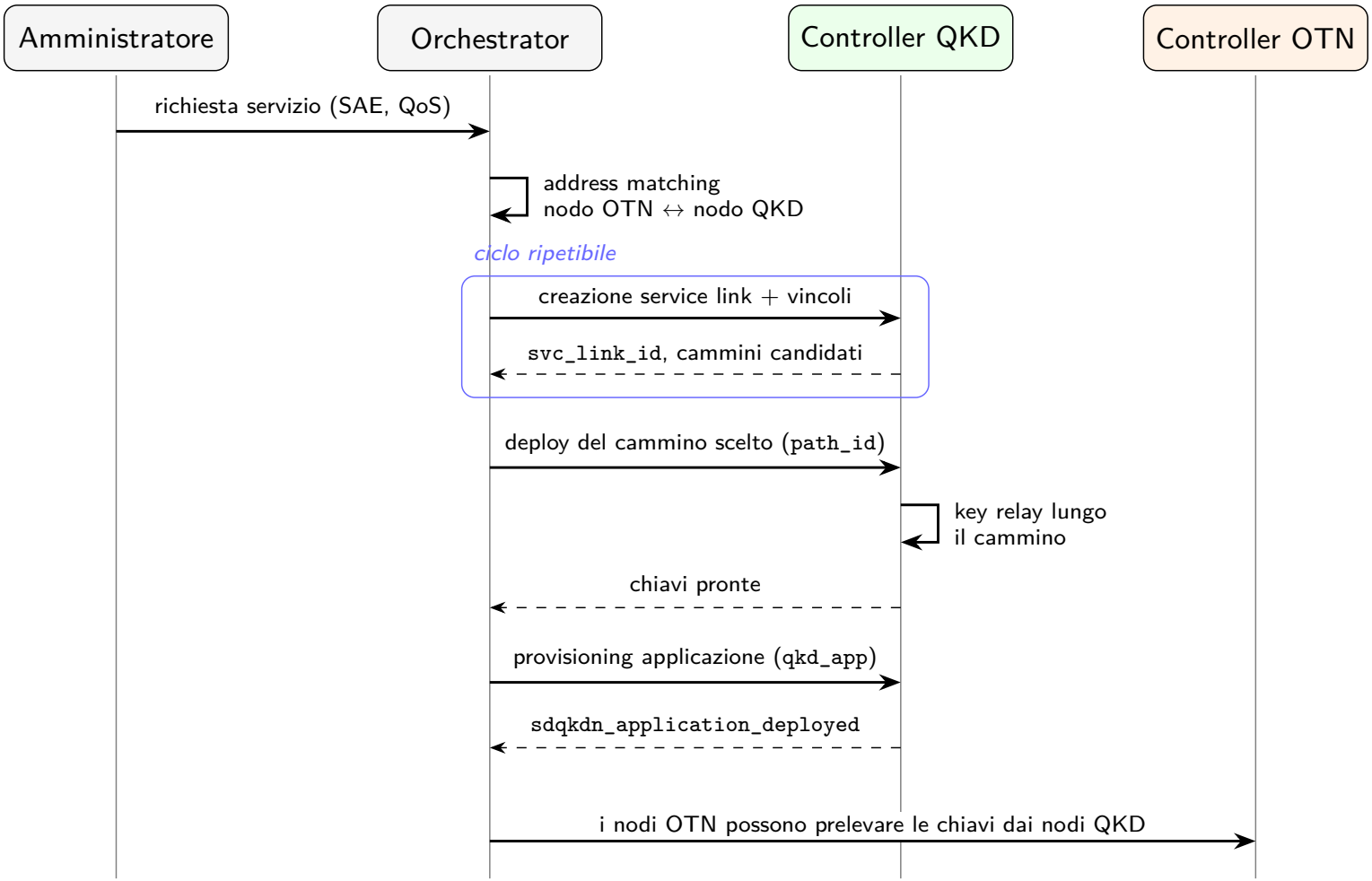
I tre moduli in sintesi

Modulo	**Domanda**	**Clausola**	**Dati**
topology	Com'è fatta la rete?	§5.1	struttura (grafo)
inventory	Quante risorse ci sono e in che stato?	§5.2, §5.5	stato, prestazioni, configurazione, notifiche
connectivity	Come servo un'applicazione end-to-end?	§5.3, §5.4, §6.2	applicazioni, RPC di calcolo e deploy dei cammini

Perché questa struttura

Chi vuole solo la struttura implementa *topology*; chi vuole anche lo stato aggiunge *inventory* via `augment`, senza duplicare nulla; *connectivity* aggiunge le operazioni.

Workflow di provisioning QKD + OTN (§6.2)



Rielaborazione semplificata di GS QKD 018, Fig. 2 [9]

Il workflow passo per passo

1. L'amministratore chiede un servizio (SAE, QoS).
2. L'orchestrator individua i controller ed effettua l'address matching.
3. Chiede al controller QKD un service link e i cammini, con vincoli.
4. Il controller assegna `svc_link_id` e restituisce candidati con priorità (passo ripetibile).
5. L'orchestrator sceglie e chiede il deploy; i nodi fanno key relay lungo il cammino.
6. Provisioning dell'app e avviso al controller OTN.

015 vs 018

Nella 015 il controller calcola e configura da solo il link virtuale; nella 018 lo propone e attende la scelta.

Più domini QKD

L'orchestrator ripete i passi 3–5 per ogni dominio lungo il cammino inter-dominio, collegando i segmenti nei nodi di bordo.

Protocolli e sicurezza

Trasporto (§8)

- Modello (YANG 1.1) separato dal protocollo: NETCONF o RESTCONF.
- RESTCONF: ogni risorsa è un URI; `GET` per leggere, `POST`/`PUT`/`PATCH`/`DELETE` per configurare; notifiche via stream.

Sicurezza (§7)

Le chiavi QKD **non transitano mai** per controller o orchestrator: nei modelli compaiono solo tassi e l'indirizzo del KME. Un orchestrator compromesso causa disservizi, non lettura delle chiavi. La protezione del canale (es. TLS) è lasciata all'implementazione.

```
GET  /restconf/data/etsi-qkd-sdn-topology:qkd_network
POST /restconf/operations/etsi-qkd-sdn-connectivity:create-service-link-id-for-app
```

La 018 applica alla QKD il pattern ACTN

ACTN	**ETSI QKD**	**Osservazione**
CNC	amministratore / portale cliente	origina la richiesta
MDSC	SDN orchestrator	vista multi-dominio, decisione del cammino
PNC	SDN controller del dominio QKD	calcola i candidati, attiva i cammini
MPI	interfaccia GS QKD 018	topologia astratta, provisioning, notifiche
SBI	interfaccia GS QKD 015	controllo dei singoli nodi
TE topology astratta	`etsi-qkd-sdn-topology`	vista vendor-neutral
banda TE	SKR / ESKR	la risorsa da allocare è la chiave

La differenza: la risorsa

In ACTN la banda è fissata dalla capacità del canale; in QKD l'SKR *fluttua* e la chiave si accumula in buffer finiti. Da qui il peso di `_perf_update`, `_overloaded` e `path_restoration`.

[24]

1 Introduzione: dalla QKD alla rete

2 ETSI GS QKD 015: il controller SDN

3 ETSI GS QKD 018: l'interfaccia di orchestrazione

4 Casi reali, limiti e conclusioni
- SK Telecom
- PoliMi e MadQCI
- Limiti
- Conclusioni

SK Telecom: orchestrazione multi-vendor su KOREN

- Primo dispiegamento documentato di un orchestrator conforme alla 018 (2020–2022) [16].
- Due operatori, KMS e dispositivi di vendor diversi, ognuno col proprio controller (QKDNC); un orchestrator (QKDNO).
- BB84 decoy state, tratte da 39 a 100 km.
- Nodi di bordo (Pangyo, Daejeon): nessun modulo QKD, due KMS interconnessi.

Estensioni al modello

- tipo di applicazione *multi-segment* (`MS`), con `ms_app_id` comune ai domini;
- `server_ms_info`, `client_ms_info`;
- RPC `app-registration-ms`;
- chiavi al bordo via GS QKD 020 [10].

`CLIENT`/`INTERNAL` non bastano per il multi-dominio; le `identity` YANG permettono di estendere.

PoliMi e MadQCI

Politecnico di Milano [17]

- Controllo gerarchico per reti metropolitane.
- Controller di dominio su ONOS con il modello 015.
- Interfaccia 018 via RESTCONF verso un orchestrator Python che aggrega più domini.
- Tutta la pila 015 $\rightarrow$ 018 in software aperto.

MadQCI, Madrid [18]

- Rete reale: decine di moduli multi-vendor, oltre 130 km, integrata con la rete ottica di produzione.
- Controllo SD-QKD basato sulla 015.
- La 018 è indicata come attività *ancora in corso*.

Il controllo di dominio è maturo, l'orchestrazione molto meno.

Limiti e prospettive

Adozione la 015 è in reti reali, la 018 quasi solo in testbed.

Multi-dominio lo scenario normativo è QKD + OTN; QKD + QKD richiede estensioni e la GS QKD 020.

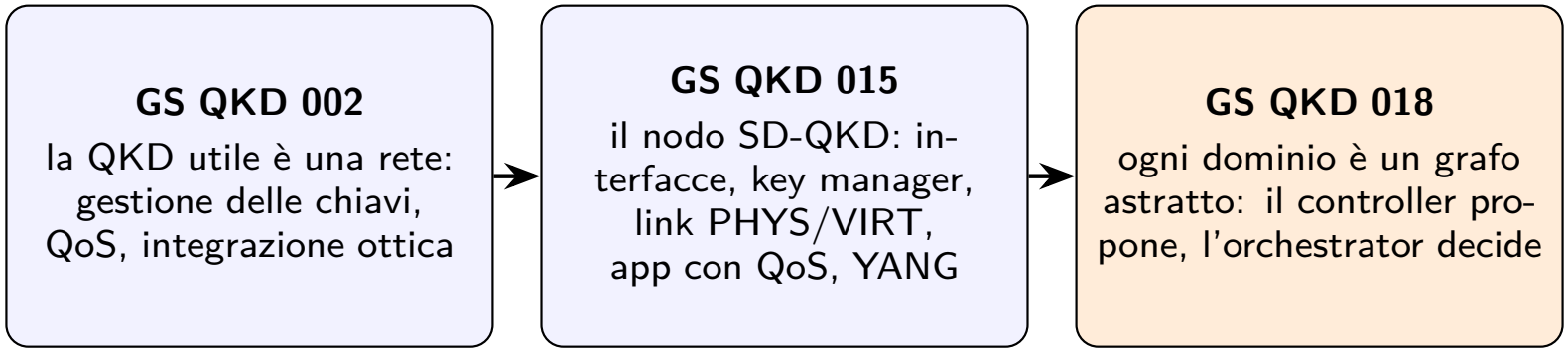
Metriche l'ESKR è istantaneo e ignora i buffer [25].

Riservatezza esporre la topologia è accettabile dentro un operatore, meno tra operatori [25, 26].

Sicurezza protezione del canale orchestrator–controller non specificata.

Algoritmi calcolo dei cammini non standardizzato [4, 27].

Conclusioni



- Il disegno ricalca ACTN e riusa YANG, NETCONF e RESTCONF.
- Principio essenziale: le chiavi non lasciano mai i nodi.
- KOREN, PoliMi e MadQCI: lo schema funziona, ma l'orchestrazione è giovane. Prossimo passo: multi-operatore, metriche migliori, riservatezza tra domini.

Riferimenti I

[1] Stefano Pirandola et al. «Advances in quantum cryptography». In: *Advances in Optics and Photonics* 12.4 (2020).

[2] Miralem Mehic et al. «Quantum Key Distribution: A Networking Perspective». In: *ACM Computing Surveys* 53.5 (2020).

[3] Yuan Cao et al. «The Evolution of Quantum Key Distribution Networks: On the Road to the Qinternet». In: *IEEE Communications Surveys & Tutorials* (2022).

[4] *QKD Routing Protocol in Quantum Networks: Overview and Challenges*. arXiv:2407.13156. 2024.

[5] ETSI ISG QKD. *Quantum Key Distribution (QKD); Use Cases*. Group Specification GS QKD 002 V1.1.1. ETSI, giu. 2010.

[6] ETSI ISG QKD. *Quantum Key Distribution (QKD); Application Interface*. Group Specification GS QKD 004 V2.1.1. ETSI, 2020.

[7] ETSI ISG QKD. *Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API*. Group Specification GS QKD 014 V1.1.1. ETSI, 2019.

[8] ETSI ISG QKD. *Quantum Key Distribution (QKD); Control Interface for Software Defined Networks*. Group Specification GS QKD 015 V2.1.1. ETSI, apr. 2022.

[9] ETSI ISG QKD. *Quantum Key Distribution (QKD); Orchestration Interface for Software Defined Networks*. Group Specification GS QKD 018 V1.1.1. ETSI, apr. 2022.

[10] ETSI ISG QKD. *Quantum Key Distribution (QKD); Interoperable Key Management System*. Group Specification GS QKD 020. ETSI.

Riferimenti II

[11] ITU-T. *Overview on networks supporting quantum key distribution*. Recommendation Y.3800. International Telecommunication Union, 2019.

[12] ITU-T. *Quantum key distribution networks – Control and management*. Recommendation Y.3804. International Telecommunication Union, 2025.

[13] ITU-T. *Quantum key distribution networks – Software-defined networking control*. Recommendation Y.3805. International Telecommunication Union, 2021.

[14] Alejandro Aguado et al. «First experimental demonstration of secure NFV orchestration over an SDN-controlled optical network with time-shared QKD resources». In: *European Conference on Optical Communication (ECOC)*. arXiv:1604.05861. 2016.

[15] Alejandro Aguado et al. «Enabling Quantum Key Distribution Networks via Software-Defined Networking». In: *International Conference on Optical Network Design and Modeling (ONDM)*. 2020.

[16] Dong-Hi Sim, Jongyoon Shin e Min Hyung Kim. «Software-Defined Networking Orchestration for Interoperable Key Management of Quantum Key Distribution Networks». In: *Entropy* 25.6 (2023), p. 943.

[17] E. Vicario. «Hierarchical SDN control for advanced quantum-secured metropolitan networks». Tesi di laurea mag. Politecnico di Milano, 2025.

[18] A. Sebastián-Lombraña et al. *A blueprint for large-scale quantum-network deployments*. arXiv:2409.01069. 2024.

[19] Diego Kreutz et al. «Software-Defined Networking: A Comprehensive Survey». In: *Proceedings of the IEEE* 103.1 (2015).

Riferimenti III

[20] Martin Bjorklund. *The YANG 1.1 Data Modeling Language*. RFC 7950. IETF, 2016.

[21] Rob Enns et al. *Network Configuration Protocol (NETCONF)*. RFC 6241. IETF, 2011.

[22] Andy Bierman, Martin Bjorklund e Kent Watsen. *RESTCONF Protocol*. RFC 8040. IETF, 2017.

[23] ETSI ISG QKD. *gs018-orch-int-sdn: YANG modules etsi-qkd-sdn-topology, etsi-qkd-sdn-inventory, etsi-qkd-sdn-connectivity (rev. 2022-02-22)*. ETSI Forge. 2022.

[24] Daniele Ceccarelli e Young Lee. *Framework for Abstraction and Control of TE Networks (ACTN)*. RFC 8453. IETF, 2018.

[25] S. Laschet et al. «SDN Key Relay for Large-Scale QKD Networks». In: *IEEE OCNC*. arXiv:2608.17539. 2026.

[26] *Interconnecting Regional QKD Networks: Hybrid Key Delivery Across Quantum Domains*. arXiv:2604.20376. 2026.

[27] R. B. Méndez et al. «Switching Coordinator: An SDN Application for Flexible QKD Networks». In: *Entropy* 28 (2026). arXiv:2603.13812, p. 219.

Grazie per l'attenzione

Domande?